

Experiment 5

Aim - To Build the pipeline of jobs using Maven / Gradle / Ant in Jenkins, create a pipeline script to Test and deploy an application over the tomcat server

Programming in Jenkins:

Continuous Integration is a software development practice where members of a team integrate their

work frequently, usually each person integrates at least daily leading to multiple integrations per day.

Each integration is verified by an automated build (including test) to detect integration errors as

quickly as possible.” In simple way, Continuous integration (CI) is the practice of frequently building

and testing each change done to your code automatically.

Jenkins is a self-contained, open-source automation server which can be used to automate all sorts of

tasks related to building, testing, and delivering or deploying software.

Our first job will execute the shell commands. The freestyle project provides enough options and

features to build the complex jobs that you will need in your projects.

Example 1

Example 1.1: Deploying a freestyle app in Jenkins

Creating a job:

Naming the job and setting it as freestyle:

Selecting build type as “Execute shell”:

Entering a simple command for the shell execution:

Applying and saving the project configuration:

Building the project:

Console output (after building):

Example 1.2: Taking parameters through files

Contents of script example1.cmd:

Executing script example1.cmd on the terminal:

Modifying the Jenkins project to execute the script while supplying required parameters:

Console output after building the modified project:

Example 2

Example 2.1: Running a Java program under Jenkins

Creating a simple Java program:

Compiling and running the program on the terminal:

Creating a new freestyle project:

Configure new project:

Console output after building:

Example 3

Example 3.1: Parameterise build

Creating a new freestyle project:

Enabling parameterisation and adding a String parameter:

Configuring the string parameter as Fname:

Adding a choice parameter and configuring it as City with the following choices:

Creating a script which takes 2 arguments for name and city:

Configuring build steps:

Entering parameters for build:

Console output after building:

Example 3.2: Running a Java program with parameters

Creating a Java program with an input argument:

Testing the program on the terminal:

Creating a new freestyle project:

Parameterise the project by adding a string parameter as follows:

Configure the build steps:

Entering the parameter for the build:

Console output after building:

Example 5

Example 5.1: Running a Python program

Creating a simple Python script:

Running the Python script on the terminal:

Creating a new freestyle project:

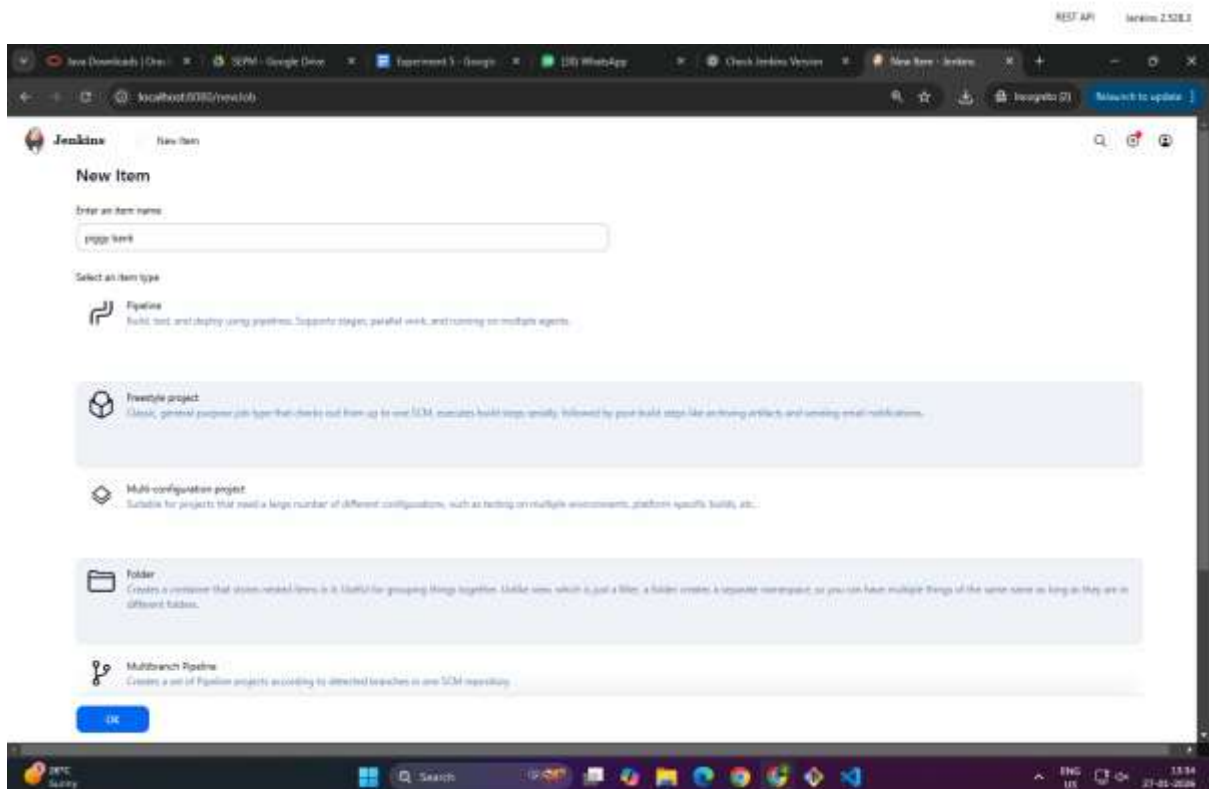
Parameterising the project with a string parameter as follows:

Configuring the build steps:

Setting the parameter for the build:

Console output after building:

Conclusion: Thus, we have successfully studied Continuous Integration and installed, configured, and understood programming with Jenkins.



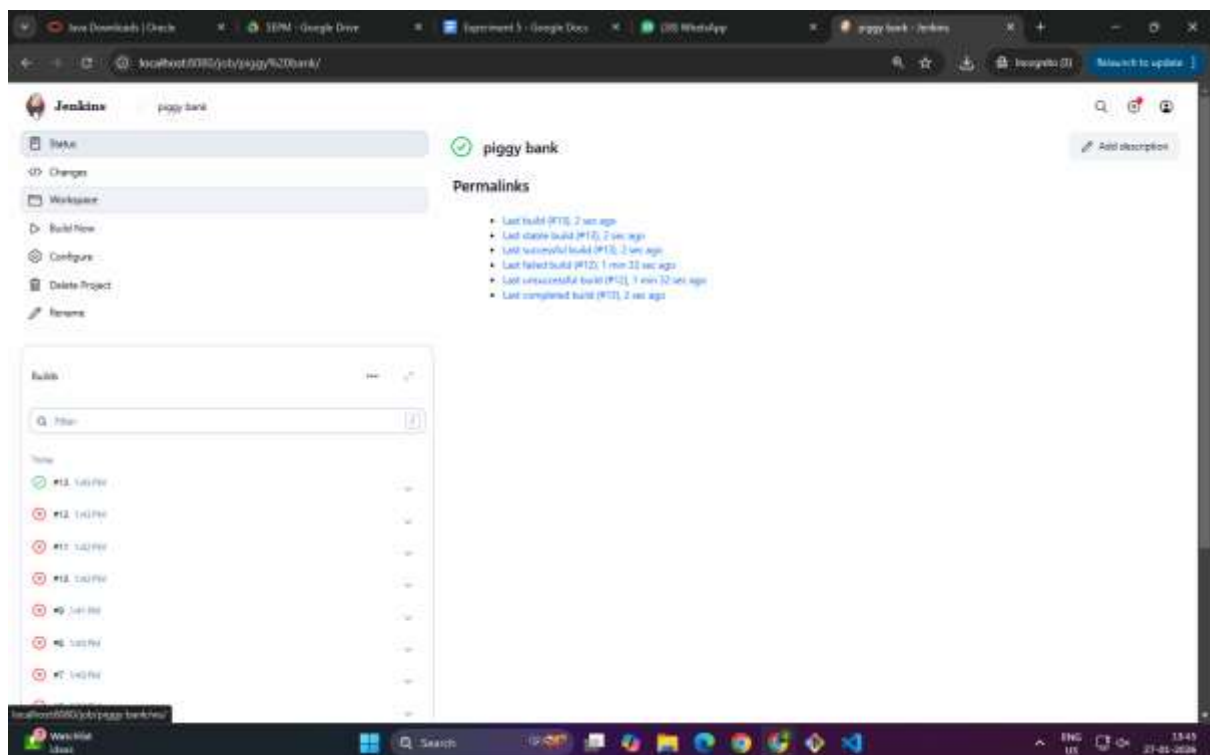
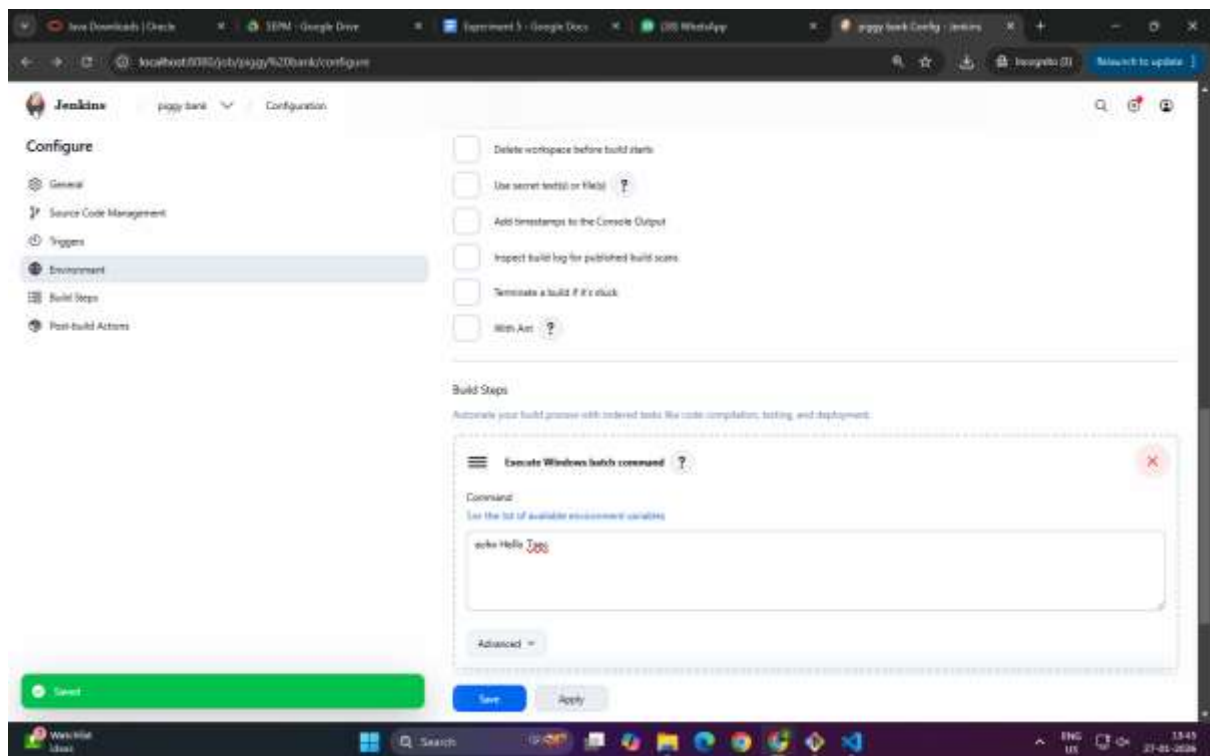
The image displays two screenshots of the Jenkins web interface, showing the configuration page for a job named 'piggy bank'.

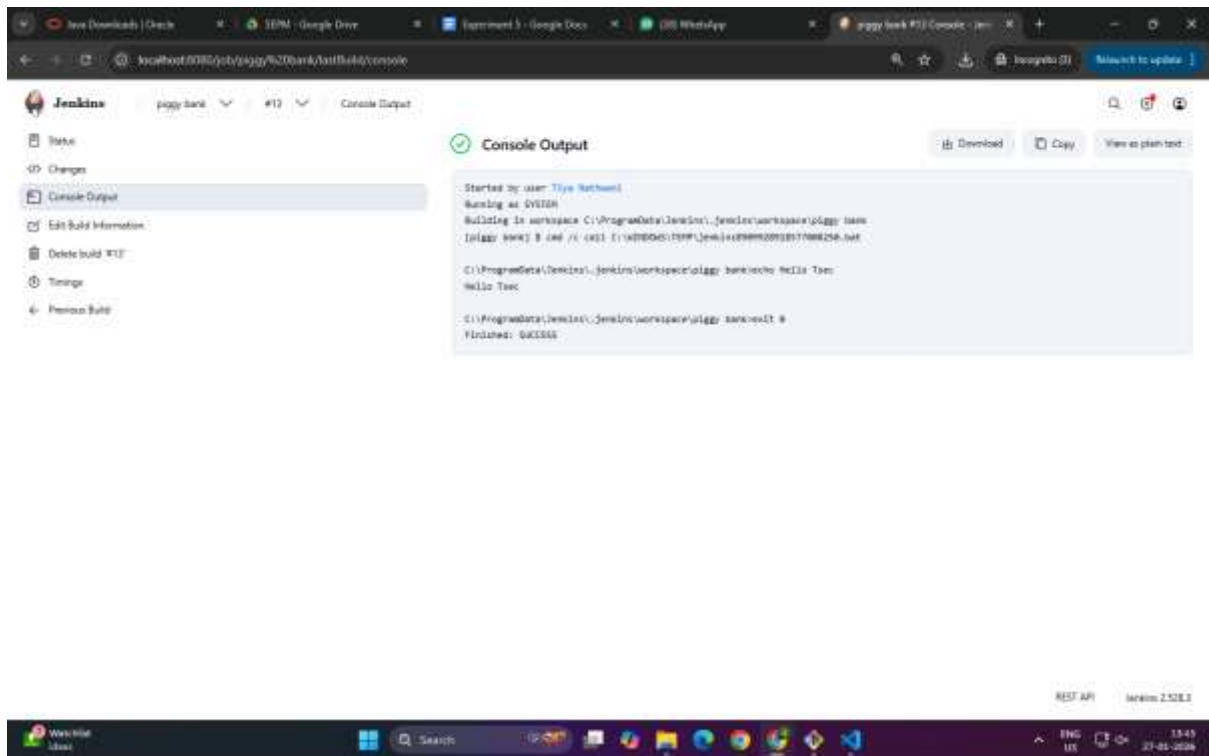
Top Screenshot: General Tab

- Configuration:** General (selected), Source Code Management, Triggers, Environment, Build Steps, Post-build Actions.
- General:** Enabled (toggle).
- Description:** Empty text area.
- Plan last:** [Previous](#)
- Options:**
 - ☐ Discard old builds ?
 - ☐ Github project
 - ☐ This project is parameterized ?
 - ☐ Throttle builds ?
 - ☐ Execute concurrent builds if necessary ?
- Advanced:** [▼](#)
- Source Code Management:** Connect and manage your code repository to automatically pull the latest code for your builds.
 - ☒ None
- Buttons:** Save, Apply

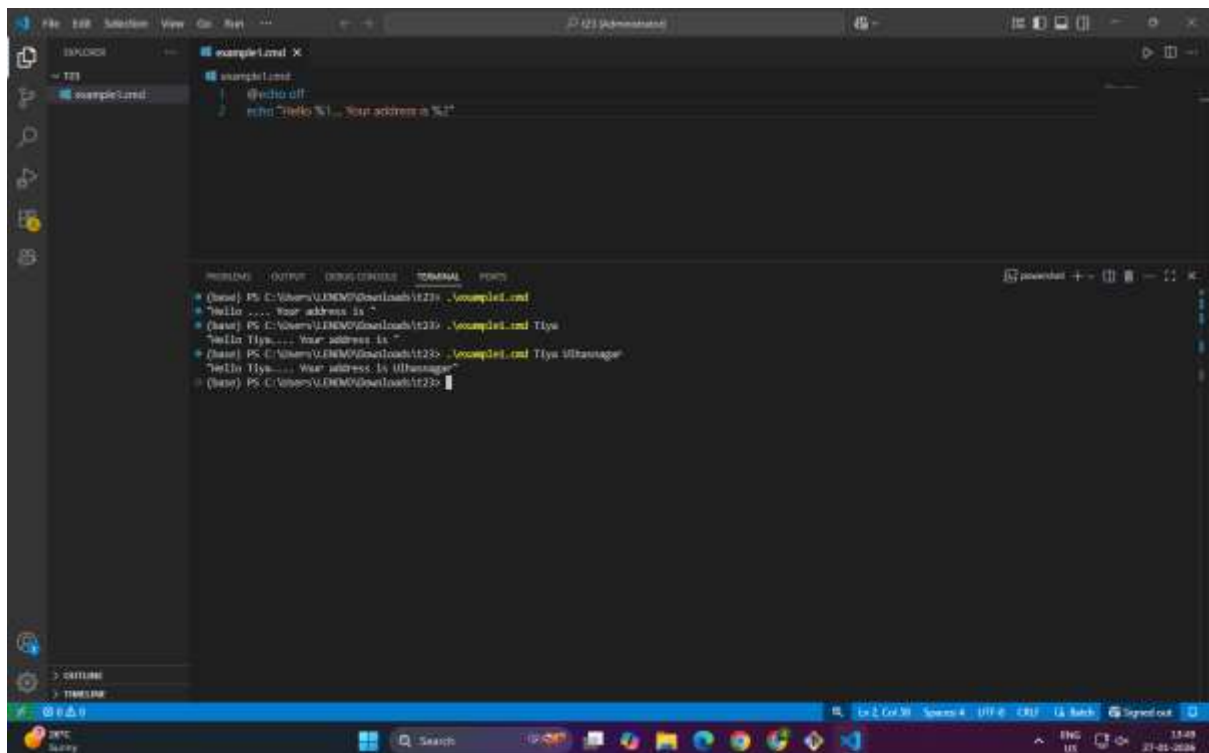
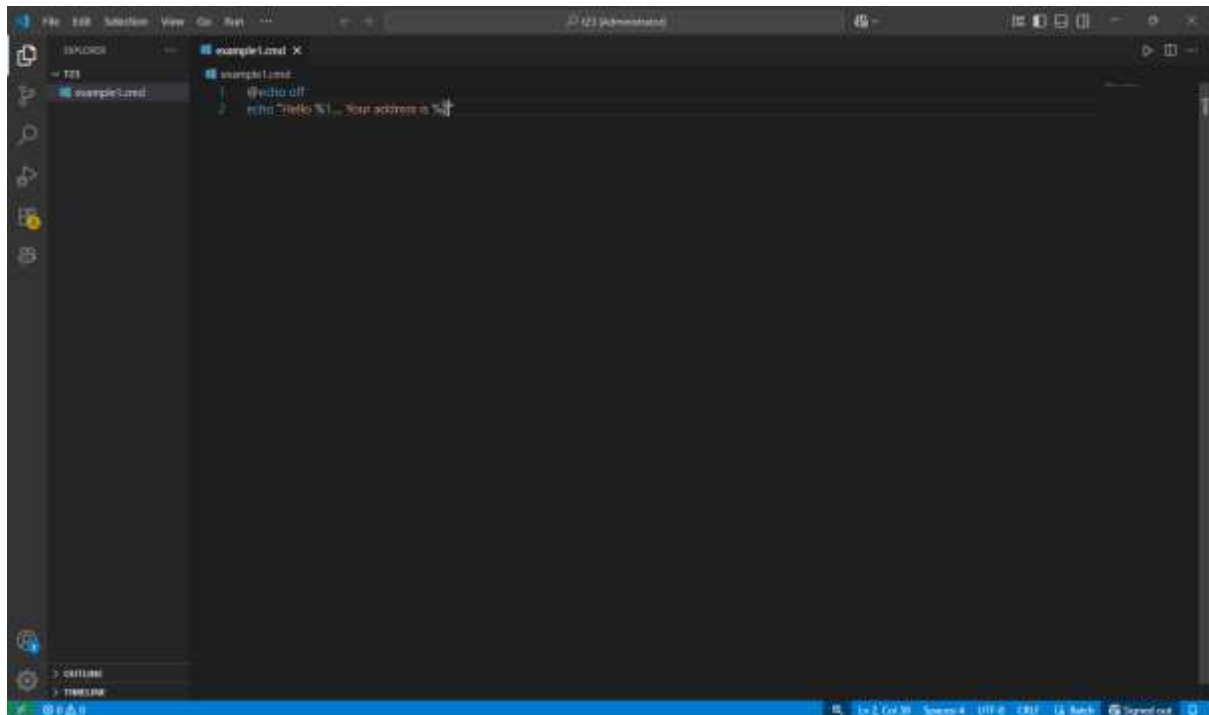
Bottom Screenshot: Build Steps Tab

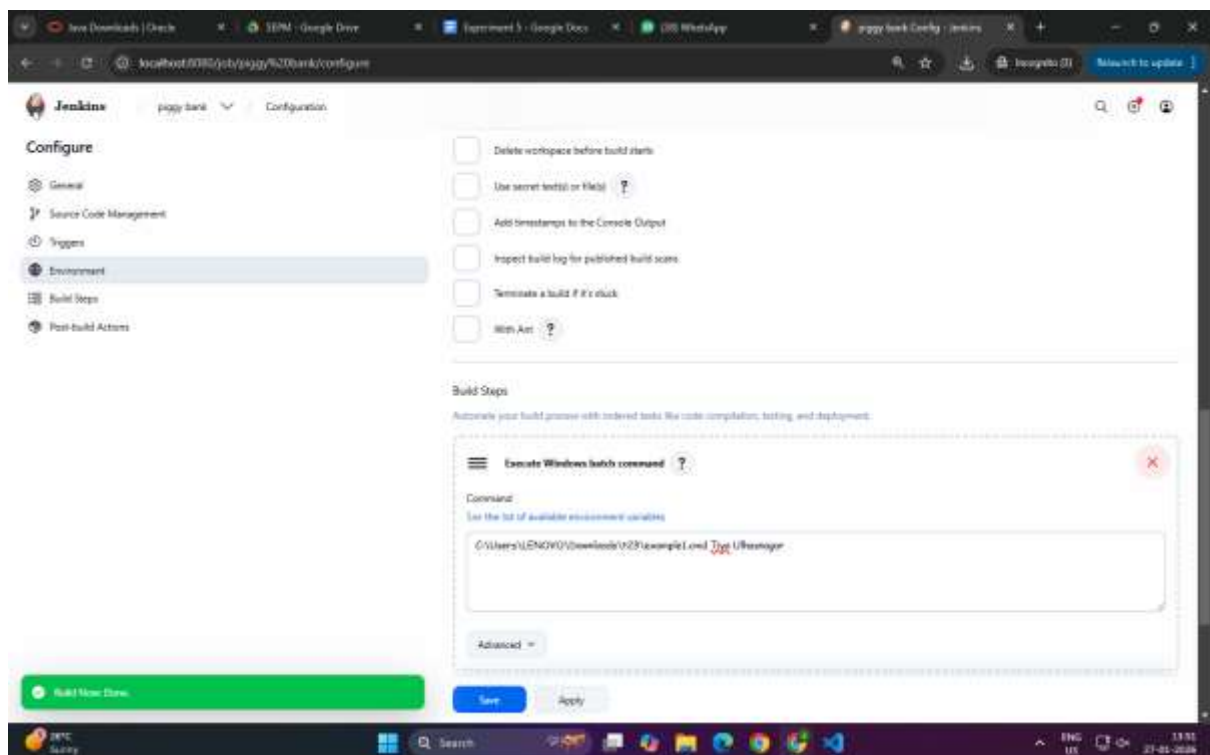
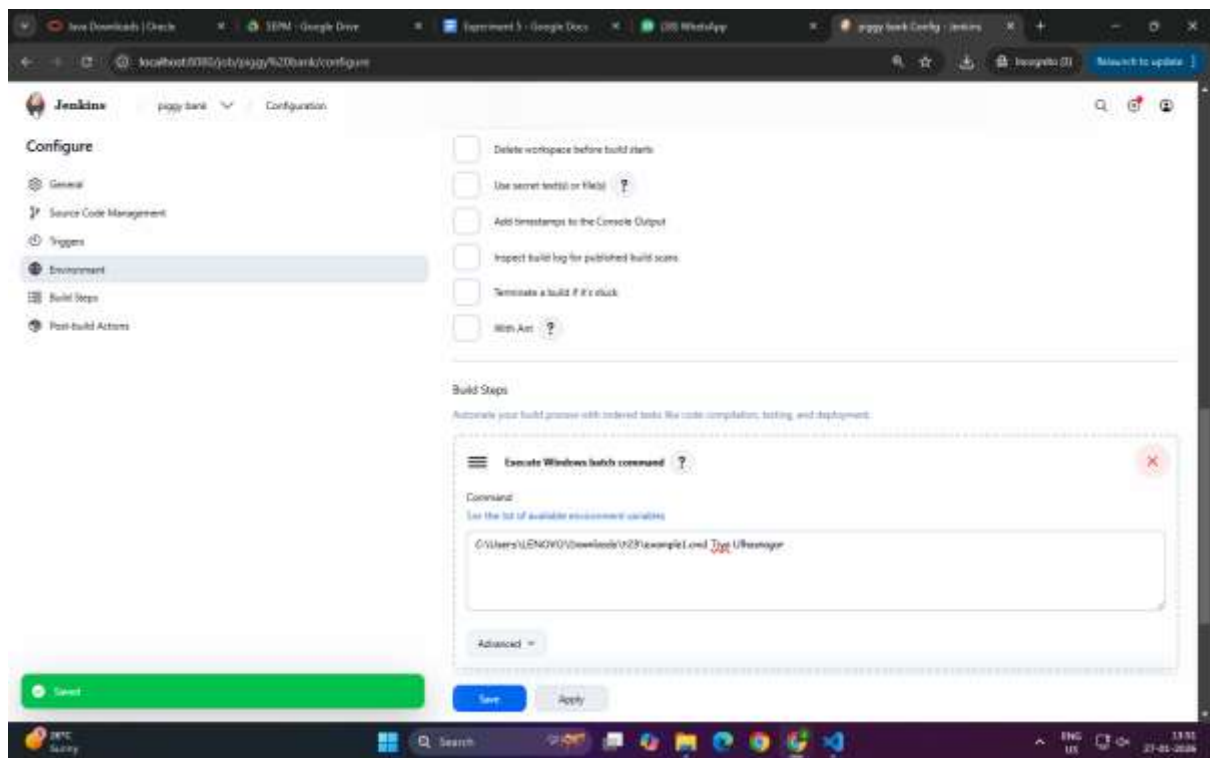
- Configuration:** General, Source Code Management, Triggers, Environment, Build Steps (selected), Post-build Actions.
- Command:** [See the list of available environment variables](#)
`echo "Jenkins file"`
- Advanced:** [▼](#)
- + Add build step**
- Post-build Actions:** Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.
 - + Add post-build action**
- Buttons:** Save, Apply
- Footer:** REST API, Jenkins 2.328.3

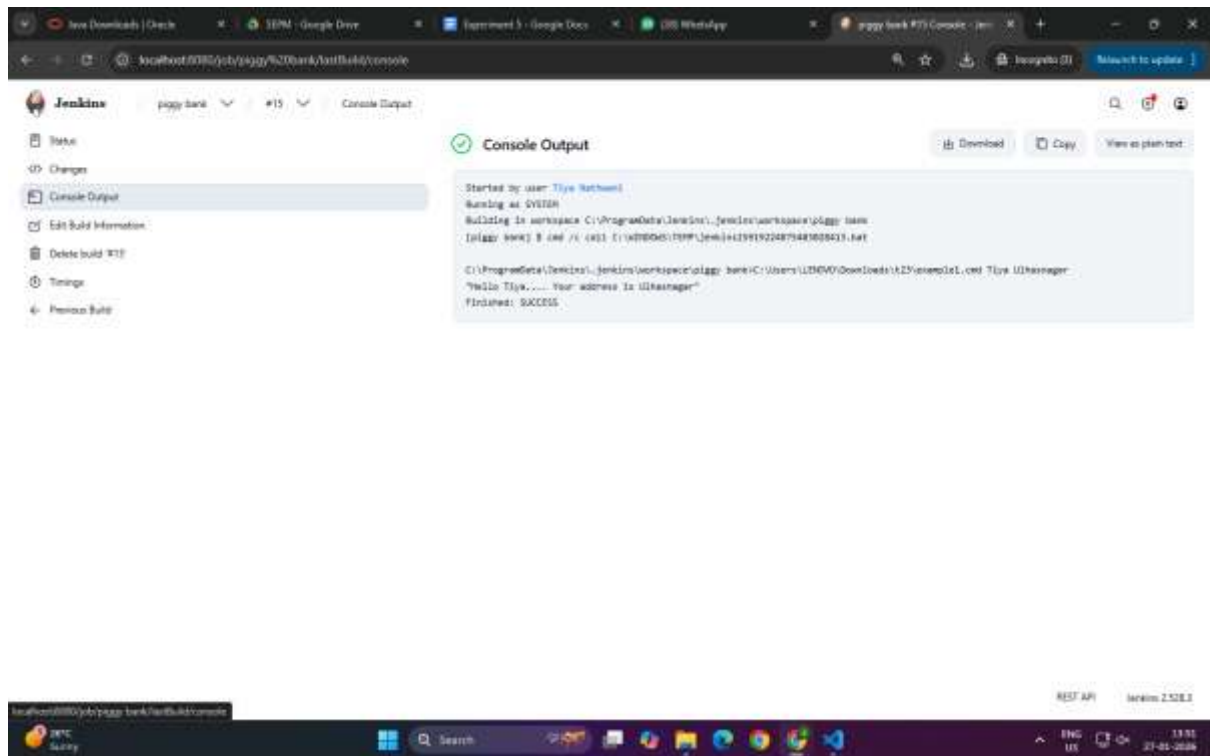




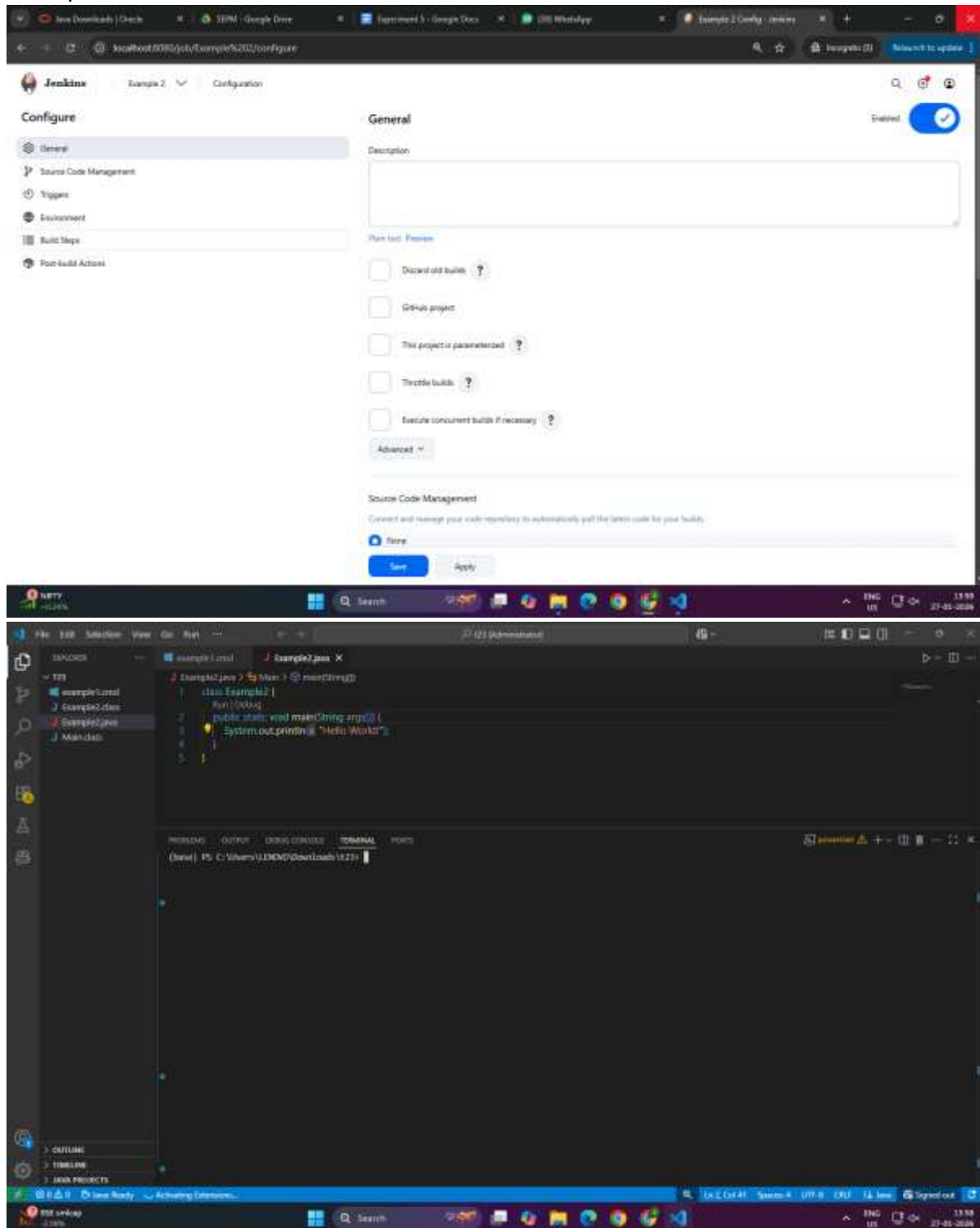
1.2







Example 2



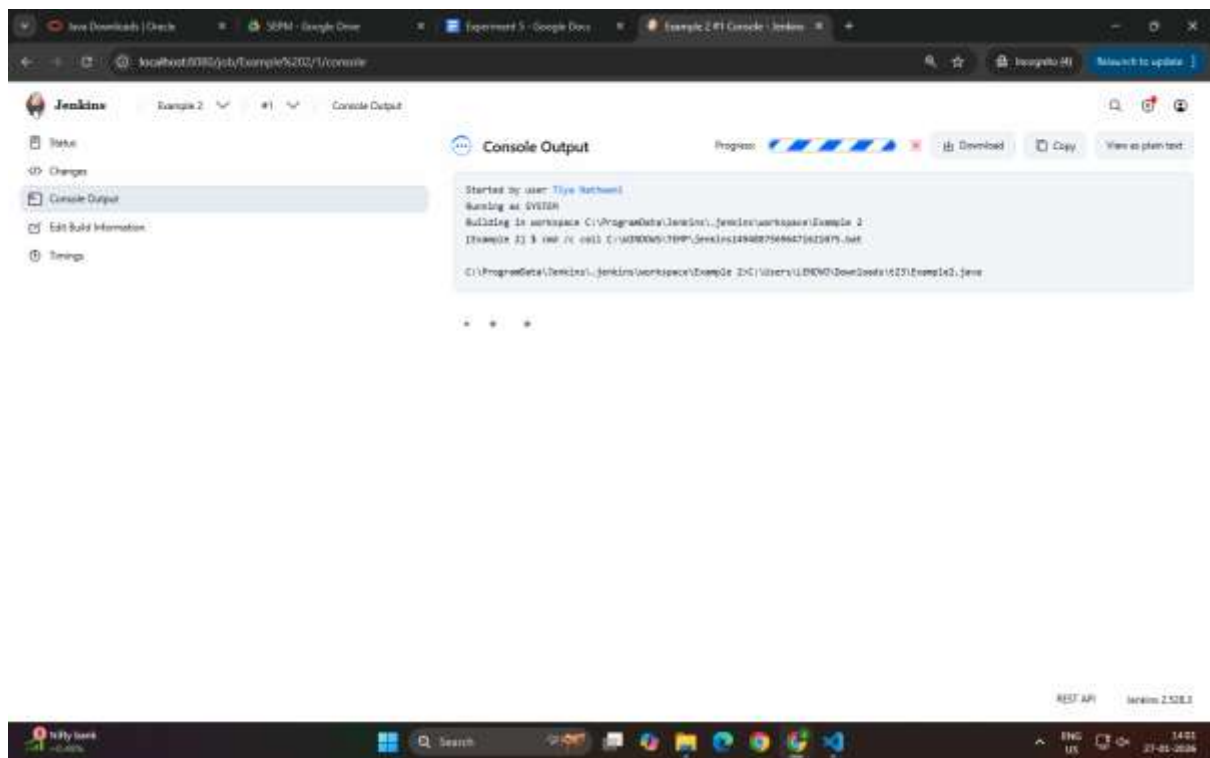
The image displays two screenshots of the Jenkins web interface, showing the configuration and overview of a build job named "Example 2".

Top Screenshot: Jenkins Configuration Page

- URL:** `localhost:8080/job/Example%202/configure`
- Left Sidebar (Configuration):**
 - General
 - Source Code Management
 - Triggers
 - Environment** (Selected)
 - Build Steps
 - Post-build Actions
- Main Content Area:**
 - General:** Includes checkboxes for "Delete workspace before build starts", "Use secret text(s) or file(s)", "Add timestamps to the console output", "Inspect build log for published build scans", "Terminate a build if it's stuck", and "Skip Ant".
 - Build Steps:** A section titled "Automate your build process with ordered tasks like code compilation, testing, and deployment." It contains a step named "Execute Windows batch command" with a command field containing `C:\Users\LENGOV\Downloads\029\Example42.java`.
- Buttons:** "Save" and "Apply" buttons are at the bottom.

Bottom Screenshot: Jenkins Overview Page

- URL:** `localhost:8080/job/Example%202/`
- Left Sidebar (Overview):**
 - Status** (Selected)
 - Changes
 - Workspace
 - Build Now
 - Configure
 - Delete Project
 - Rename
- Main Content Area:**
 - Example 2 Permalinks:** A section for managing the job's name and description.
 - Builds:** A section showing the build history, including a recent build with a status of "1: 200 ms".
- Bottom Bar:** A green bar indicating "Build scheduled".



The screenshot displays the Jenkins web interface. The top section, titled 'New Item', shows the process of creating a new Jenkins item. The name 'Example 3' is entered, and 'Pipeline' is selected as the item type. Below this, several project types are listed: 'Freestyle project', 'Multi-configuration project', 'Folder', and 'Multibranch Pipeline'. The 'Pipeline' option is highlighted.

The bottom section shows the 'Configure' page for 'Example 3'. The 'General' tab is active. The checkbox 'This project is parameterized' is checked. A dropdown menu is open, showing options for adding parameters: 'String Parameter', 'File Parameter', 'Boolean Parameter', 'Choice Parameter', 'Credential Parameter', 'Multi-line String Parameter', 'Password Parameter', and 'Run Parameter'. The 'String Parameter' option is selected.

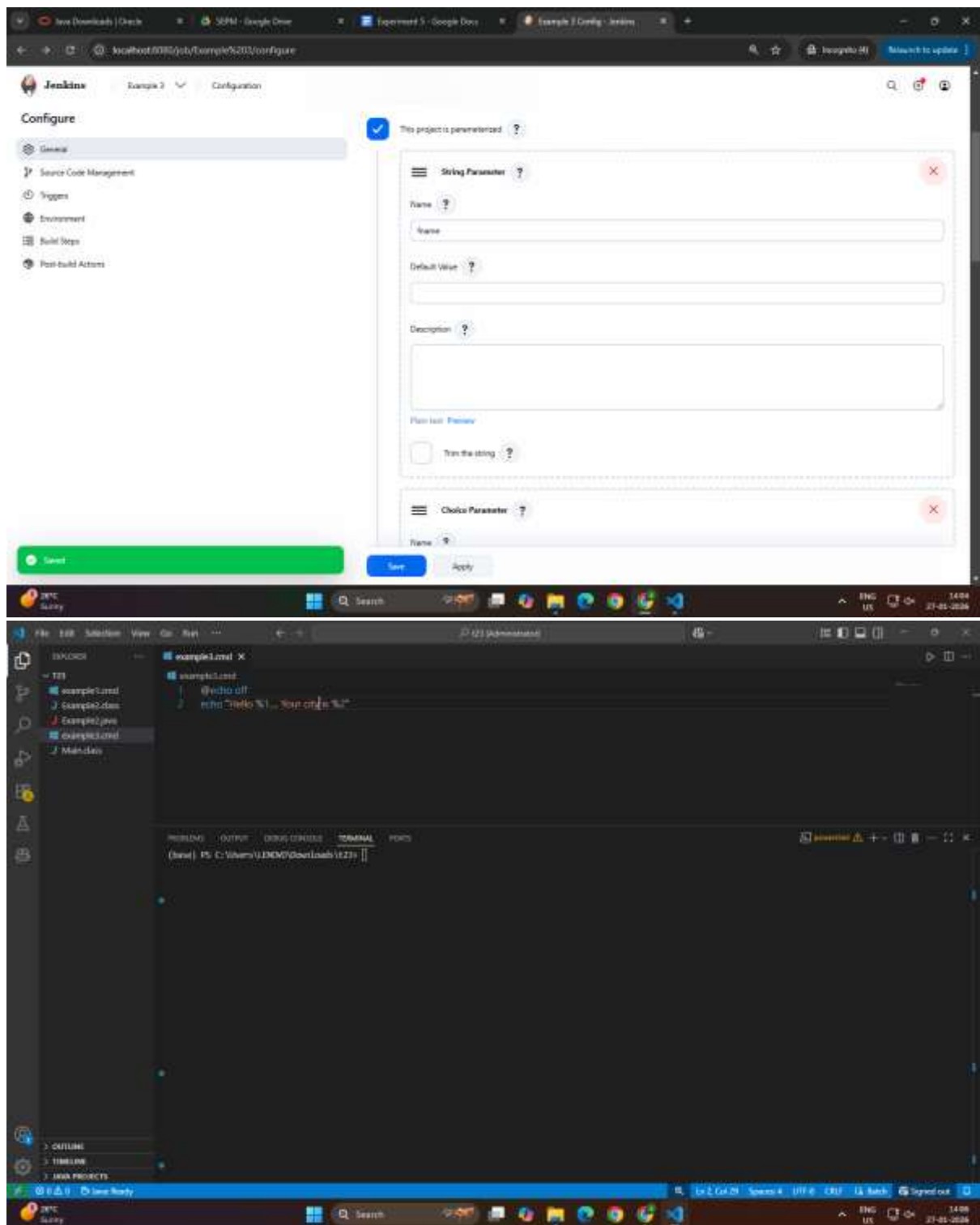
The image displays two screenshots of the Jenkins web interface, specifically the configuration page for a project named 'Example 3'. The browser address bar shows the URL `localhost:8080/job/Example%203/configure`.

Top Screenshot: String Parameter Configuration

- The left sidebar shows the 'Configure' menu with options: General, Source Code Management, Triggers, Environment, Build Steps, and Post-build Actions.
- The main content area has a blue checkmark icon and the text 'This project is parameterized.'.
- A 'String Parameter' form is visible, with fields for 'Name' (containing 'name'), 'Default Value' (empty), and 'Description' (empty).
- Below the form is a checkbox labeled 'Trim the string'.
- At the bottom are 'Save' and 'Apply' buttons.

Bottom Screenshot: Choice Parameter Configuration

- The left sidebar is the same as in the top screenshot.
- The main content area shows a 'Choice Parameter' form.
- The 'Name' field contains 'City'.
- The 'Choices' field contains a list of items: 'Bando', 'Brennager', 'Kaiser', and 'Makind'.
- The 'Description' field is empty.
- At the bottom are 'Save' and 'Apply' buttons.



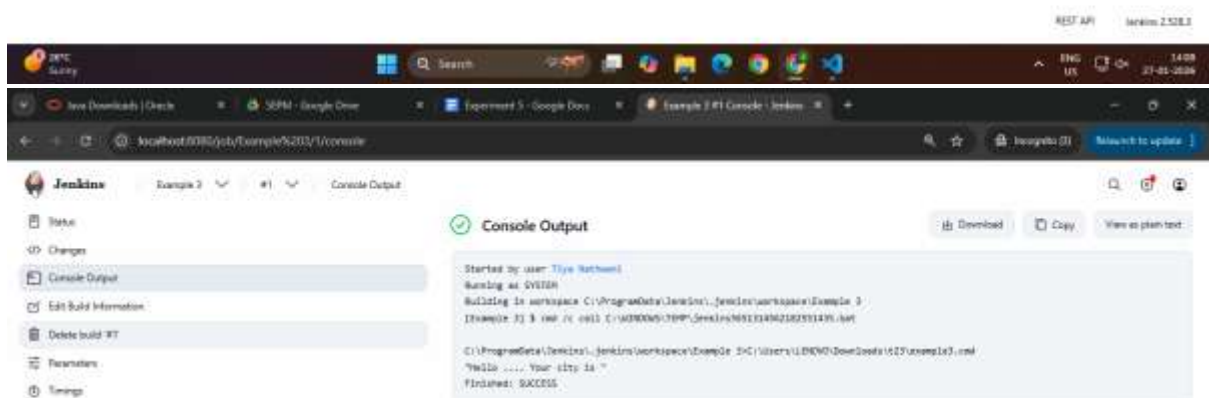
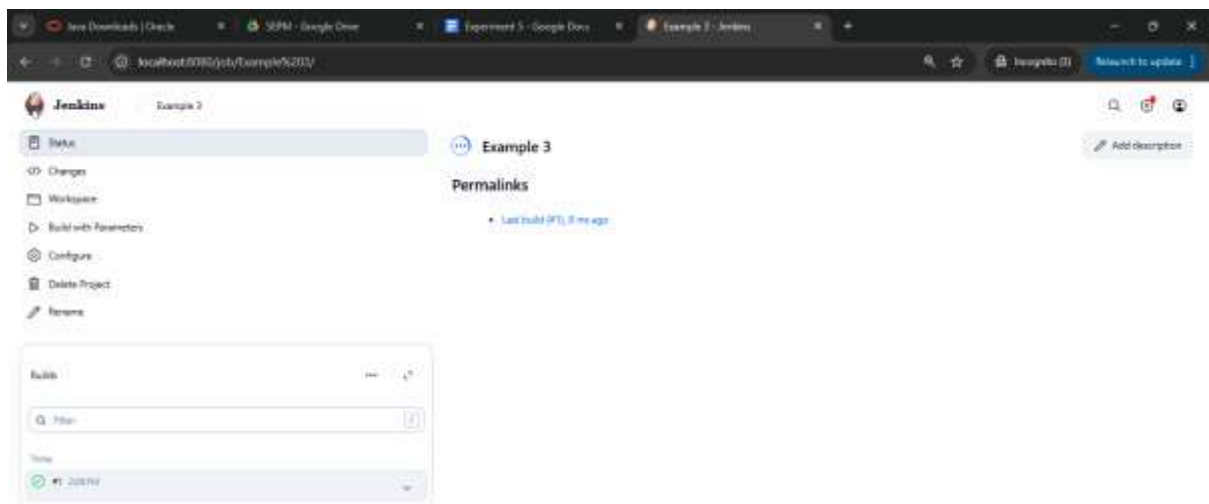
The image displays two screenshots of the Jenkins web interface, showing the configuration and build process for a project named 'Example 3'.

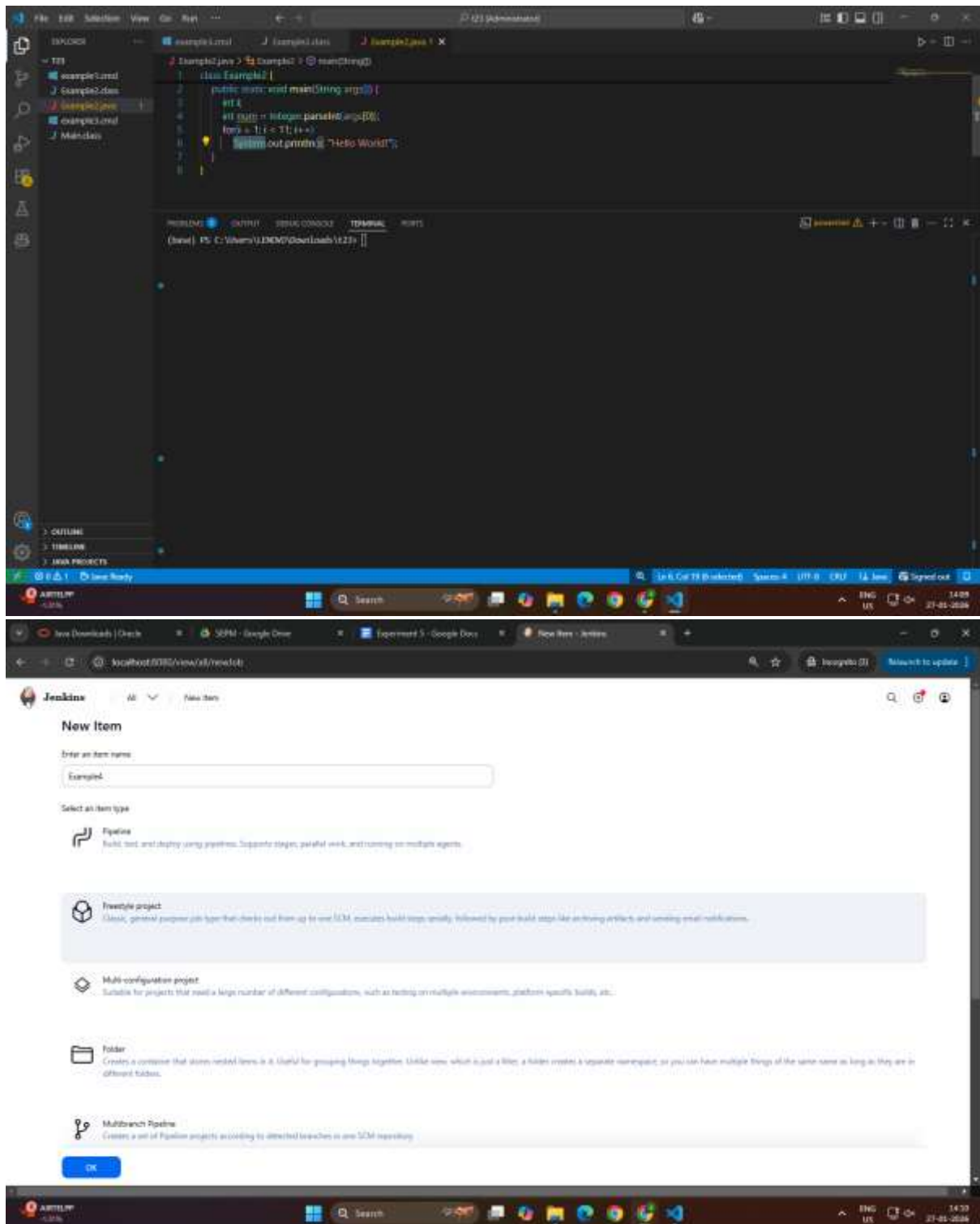
Top Screenshot: Jenkins Configuration Page

- URL:** `localhost:8080/job/Example%203/configure`
- Page Title:** Jenkins - Example 3 - Configuration
- Left Sidebar (Configure):**
 - General
 - Source Code Management
 - Triggers
 - Environment
 - Build Steps** (Selected)
 - Post-build Actions
- Build Steps Section:**
 - Execute Windows batch command** (Step 1)
 - Command:** `C:\Users\LSNGWD\Downloads\123\example3.cmd`
 - Advanced:** (Collapsed)
 - + Add build step**
- Post-build Actions Section:**
 - + Add post-build action**
- Buttons:** Save, Apply
- Footer:** REST API, Jenkins 2.528.3

Bottom Screenshot: Jenkins Build Page

- URL:** `localhost:8080/job/Example%203/build?delay=0sec`
- Page Title:** Jenkins - Example 3
- Left Sidebar (Project Example 3):**
 - Status
 - Changes
 - Workspace
 - Build with Parameters
 - Configure
 - Delete Project
 - Rename
- Build Parameters:**
 - Name:** 123
 - City:** Sandra
- Buttons:** Build, Cancel
- Build History:** No builds
- Footer:** Jenkins 2.528.3





The image displays two screenshots of the Jenkins configuration interface, showing the configuration of a Jenkins job named 'Example1'.

Top Screenshot: String Parameter Configuration

- The 'Configure' page is open, showing the 'General' tab.
- The 'This project is parameterized' checkbox is checked.
- A 'String Parameter' dialog is open, showing the configuration for a parameter named 'num'.
- The 'Name' field is set to 'num'.
- The 'Default Value' field is empty.
- The 'Description' field is empty.
- The 'Plan test' button is visible.
- The 'Add Parameter' button is at the bottom.

Bottom Screenshot: Build Steps Configuration

- The 'Configure' page is open, showing the 'Build Steps' tab.
- The 'Build Steps' section is titled 'Automate your build process with ordered tasks like code compilation, testing, and deployment'.
- A 'Execute Windows batch command' step is added, with the command field containing the text: `C:\Users\LSNGW\Downloads\123\Example1.jar %num%`.
- The 'Advanced' button is visible below the command field.
- The 'Add build step' button is at the bottom.
- The 'Post-build Actions' section is titled 'Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs'.
- The 'Add post-build action' button is at the bottom.
- The 'Save' and 'Apply' buttons are at the bottom.

